Experiment 9: Pointers

Objective: To understand and implement pointer concepts such as address manipulation, pointer arithmetic, and pointer-based functions.

Questions:

1. **Write a C program to display the value of a variable and its address using a pointer.**
   Sample Input:
   ```
   Enter a number: 10
   ```
   Sample Output:
   ```
   Value = 10
   Address = 6422296
   ```

2. **Write a C program to swap two numbers using call by reference (pointers).**
   Sample Input:
   ```
   Enter two numbers: 5 10
   ```
   Sample Output:
   ```
   After swapping: 10 5
   ```

3. **Write a C program to find the length of a string using pointer traversal.**
   Sample Input:
   ```
   Enter string: pointer
   ```
   Sample Output:
   ```
   Length = 7
   ```

4. **Write a program in C to print a string in reverse using a pointer.**
   Sample Input:
   ```
   Input a string : hello
   ```
   Sample Output:
   ```
   Reverse of the string is : olleh
   ```

5. **Write a function in C to find out the largest among two numbers. The function return type should be void and the output should be printed in the main().**
   Sample Input:
   ```
   Enter two numbers: 5  10
   ```
   Sample Output:
   ```
   The largest number is 10.
   ```

6. **Predict the output of the following code snippets. Also, justify the answers.**

a. **int a = 5;** **int *p = &a;** **int **q = &p;** ****q = **q + 10;** **printf("%d", a);**	**b.** **int a = 10;** **int *p = &a;** **int **q = &p;** ***p = 20;** ****q = 30;** **printf("%d", a);**
c. **int a[] = {10, 20, 30, 40};** **int *p = a;** **printf("%d %d %d", *p, *(p+1), *(p+3));**	**d.** **int a = 5;** **int *p = &a;** **int **q = &p;** **printf("%d", *p + **q);**

e.
`int a = 5, b = 10;`
`int *p = &a;`
`int **q = &p;`
`*q = &b;`
`printf("%d", **q);`
	f.
	`int a = 5;`
	`int *p = &a;`
	`int **q = &p;`
	`printf("%d %d %d", a, *p, **q);`

g.
`int a = 3;`
`int *p = &a;`
`int **q = &p;`
`*p = *p + 2;`
`**q = **q + 3;`
`printf("%d", a);`
	h.
	`int a = 2;`
	`int *p = &a;`
	`int **q = &p;`
	`(*p)++;`
	`(**q)++;`
	`printf("%d", a);`

i.
`int a[] = {5, 10, 15, 20};`
`int *p = a;`
`printf("%d %d", p[2], 2[p]);`
	j.
	`int a[] = {1, 2, 3, 4};`
	`int *p = a;`
	`printf("%d %d %d", *p, *p++,*++p);`

k.
`int a[] = {10, 20, 30, 40};`
`int *p = a;`
`p = p + 2;`
`printf("%d %d", *p, *(p-1));`
	l.
	`int a[] = {2, 4, 6, 8};`
	`int *p = a;`
	`printf("%d %d", *p++, (*p)++);`
	`printf(" %d", *p);`

m.
`int a[] = {10, 20, 30, 40, 50};`
`int *p = &a[1];`
`int *q = &a[4];`
`printf("%ld", q - p);`
	n.
	`int a[] = {5, 10, 15};`
	`int *p = a;`
	`int **q = &p;`
	`printf("%d %d", *p, **q);`

o.
`int a[] = {1, 2, 3};`
`int b[] = {4, 5, 6};`
`int *p = a;`
`p = b;`
`printf("%d %d", *p, *(p+2));`
	p.
	`int a[] = {1, 2, 3, 4};`
	`int *p = a;`
	`printf("%d", *(p + *(p+1)));`

q.
`int a[] = {2, 4, 6, 8};`
`int *p = a;`
`for(int i = 0; i < 4; i++)`
`printf("%d ", *(p+i));`
	r.
	`int a[] = {10, 20, 30};`
	`int *p = a;`
	`printf("%d %d", *p, *++p);`
	`printf(" %d", *p++);`
	`printf(" %d", *p);`

s.
`int a[] = {1, 3, 5, 7};`
`int *p = a + 3;`
`printf("%d %d", *p, *(p-2));`
	t.
	`int a[] = {2, 0, 5};`
	`int *p = a;`
	`if (*p && *++p || *p++)`
	`printf("%d %d", *p, *(p-1));`

u.
`int a[] = {1, 2, 3, 4};`
`int *p = a;`
`printf("%d", *(p + *(p + 2)));`
	v.
	`int a[] = {5, 10, 15, 20};`
	`int *p = a + 3;`
	`printf("%d %d", *p--, *--p);`